

Univerzita Karlova
Prírodovedecká fakulta



Aproximácia krivky metódou Chaikin

Dokumentácia k programu na predmet:
Úvod do programovania (MZ370P19)

Anna Fajtová
2. B-SGG

Vavrečka 2026

Obsah

Úvod a stručné zadanie	3
Presné zadanie	3
Popis problému	3
Zvolený algoritmus	3
Program	4
Problematické situácie a ich ošetrenia	5
Alternatívne riešenia úlohy	6
Vstupné dáta	6
Výstupné dáta	6
Priebeh práce	6
Námety na vylepšenie	7
Záverečný povzdych	7
Zoznam literatúry	7

Úvod a stručné zadanie

Tento dokument obsahuje priebeh a technické detaily práce na programe pre aplikáciu Chaikinovho algoritmu pre vyhladzovanie kriviek. Program je vytvorený v Pythone a využíva princípy objektovo orientovaného programovania (OOP). Hlavným cieľom je vytvorenie nástroja, ktorý premení textový súbor s 2D súradnicami krivky na textový súbor so súradnicami aproximovanej, a teda zjemnenej verzie danej krivky.

Presné zadanie

Z textového súboru načítajte pravouhlé súradnice lomových bodov polylinie/polygónu. Pre zadané n odpovedajúce počtu iterácií aplikujte metódu corner cutting (Chaikin algorithm). Vyhladenú líniu uložte do textového súboru.

Popis problému

Chaikinov algoritmus bol prvýkrát popísaný v roku 1974 Georgeom Chaikinom. Hlavným cieľom je postupné zjemňovanie vstupných kriviek a ich opakovaná premena na čoraz hladšiu verziu. Je to takzvaný geometrický algoritmus, ktorý generuje nový polygón orezávaním rohov z toho pôvodného, preto sa tiež niekedy nazýva ako metóda corner cutting. Základná myšlienka je, že každá úsečka tvorená bodmi P_i a P_{i+1} sa v každom kroku nahradí dvoma novými bodmi Q_i a R_i . To docieli odstrihnutie pôvodného vrcholu a jeho nahradenie kratšou a jemnejšou hranou. Body sa umiestňujú do prvej a tretej štvrtiny pôvodnej úsečky. Výsledná krivka tak neprechádza pôvodnými bodmi, ale len kopíruje ich tvar. (Chaikin 1974)

$$Q_i = 0.75 * P_i + 0.25 * P_{i+1}$$

$$R_i = 0.25 * P_i + 0.75 * P_{i+1}$$

Zvolený algoritmus

V tomto prípade nebolo nad algoritmom moc premýšľania, keďže zadanie to definuje celkom presne. Algoritmus v programe najprv prejde vstupné body, identifikuje úsečky tvorenú za sebou nasledujúcimi bodmi a potom každý tento úsek nahradí novými bodmi v $1/4$ a $3/4$ dĺžky pôvodnej úsečky. Následne sa tento proces zopakuje vždy na novovytvorených bodoch toľkokrát, koľko iterácií užívateľ zadal. Počet možných iterácií bol limitovaný na max. 15, pretože počet bodov pri väčších opakovaníach rastie šialene rýchlo a výstupné súbory by boli priveľmi dlhé. V prípade, že algoritmus rozozná rovnaký začiatkový a koncový bod vo vstupe, tak dopočíta spojenie (resp. skopíruje prvý bod aj na koniec) medzi prvým a posledným novovzniknutým bodom vo výstupe aby sa zachovala vždy rovnaká geometria.

Program

Štruktúra programu začína triedou *Point*. V nej sa pomocou konštruktóra `__init__` inicializujú súradnice *x* a *y* ako desatinné čísla a definuje sa metóda `__eq__`, ktorá porovnáva identity bodov. Trieda *Polyline* následne obsahuje metódu `closed()` kde sa vyhodnotí uzavrenosť krivky podľa porovnanie jej prvého a posledného bodu. Trieda *Polygon* dedí z triedy *Polyline*, ale na rozdiel od nej vždy pri uzavrenosti vracia hodnotu *True*, keďže polygón je vždy uzavretý útvar.

Hlavný výpočet je pod triedou *Chaikin*. Jedna iterácia Chaikinovho algoritmu podľa vzorcov je spísaná pod metódou `corner_cutting()`, a následne metóda `run()` zabezpečuje cyklus postupného zjemňovania podľa zadaného počtu iterácií.

Hlavný blok kódu začne najprv otvorením a riadkovým spracovaním vstupného súboru, kde prefiltruje prázdne znaky a prevedie vstup na objekty typu *Point*. Po prejdení rôznych ošetrovaní program vytvorí inštanciu geometrie a spustí algoritmus. Nakoniec sa všetko zapíše v rovnakom formáte ako vstup do nového textového súboru s názvom „*chaikin.txt*“.

```
1  #Aproximácia krivky metódou Chaikin
2  #Anna Fajtová, 2. ročník, 8-SGG
3  #zimný semester 2025/2026
4  #Úvod do programovania M2370P19
5
6  class Point: #bod v 2D
7      def __init__(self, x, y): #konštruktör
8          self.x = float(x)
9          self.y = float(y)
10     def __eq__(self, other): #porovnávanie či sa dva body rovnajú
11         if not isinstance(other, Point):
12             return False
13         return (self.x, self.y) == (other.x, other.y)
14
15     class Polyline: #trieda pre polyliniu
16         def __init__(self, points):
17             self.points = points
18         def closed(self): #rozlíšenie v uzavrenosti medzi polyliniou a polygónom
19             if len(self.points) >= 2 and self.points[0] == self.points[-1]:
20                 return True
21             return False
22
23     class Polygon(Polyline): #trieda pre polygón, dedí od polylinie
24         def closed(self): #polygón je vždy uzavretý
25             return True
26
27     class Chaikin: #trieda pre výpočty
28         def __init__(self, geometry):
29             self.geometry = geometry
30         def corner_cutting(self): #jedna iterácia vyhladenia krivky
31             p = self.geometry.points
32             new = [] #zoznam pre novovzniknuté body
33             for i in range(len(p) - 1):
34                 p0, p1 = p[i], p[i+1] #začiatok a koniec úsečky
35
36                 q_x = 0.75 * p0.x + 0.25 * p1.x #vzorce podľa chaikina
37                 q_y = 0.75 * p0.y + 0.25 * p1.y
38                 new.append(Point(q_x, q_y))
39                 r_x = 0.25 * p0.x + 0.75 * p1.x
40                 r_y = 0.25 * p0.y + 0.75 * p1.y
41                 new.append(Point(r_x, r_y))
42
43             if self.geometry.closed(): #ak je to polygón, musia sa prepojiť nové body
44                 first = new[0]
45                 new.append(Point(first.x, first.y))
46
47             self.geometry.points = new
48
49         def run(self, n): #vyhladenie pre zadané n iterácií
50             if 0 <= n <= 15:
51                 for _ in range(n):
52                     self.corner_cutting()
53                 return True
54             print("Chyba - rozsah iterácií musí byť 0 až 15 :(")
55             return False
```

```

66 points = []
67
68 try:
69     with open("points.txt", "r") as f:
70         for line in f:
71             if not line.strip(): #preskočenie prázdnych riadkov
72                 continue
73             u = line.replace("[", "").replace("]", "").replace(" ", "").split()
74             if len(u) != 2: soietrenie aby z riadku boli iba dve suradnice
75                 print("ajaj, neplatný formát dát, skús to znova")
76                 points = []
77                 break
78             try:
79                 points.append(Point(u[0], u[1]))
80             except ValueError: #soietrenie neplatného formátu dát
81                 print("ajaj, neplatný formát dát, skús to znova")
82                 points = []
83                 break
84 except FileNotFoundError: #soietrenie neexistencie súboru
85     print("Neexistujúca krivka v neexistujúcom súbore sa ešte bohužiaľ vyhladiť nedá :(")
86
87 if len(points) >= 2:
88     try:
89         n = int(input("Počet iterácií: "))
90         geometry = Polyline(points) #inštancia polyline
91         i = Chaikin(geometry) #inicializácia
92         if i.run(n):
93             with open("chaikin.txt", "w") as f:
94                 for p in geometry.points: #zápis bodov v rovnakom tvare ako vstup
95                     f.write(f"{p.x},{p.y}\n")
96             print("Podarilo sa!!! Výsledok je v súbore chaikin.txt")
97         except ValueError: #soietrenie zlého zadania počtu iterácií
98             print("Chyba - iterácia musí byť celé číslo")
99     elif len(points) == 1: #ak je menej než 2 body
100         print("Nedostatok bodov na spracovanie :(")

```

Problematické situácie a ich ošetrenia

- A. Problém: Chaikinov algoritmus zjemňuje hrany, teda logicky by nemohol fungovať v prípade, že by vo vstupných dátach boli menej ako 2 body.
Riešenie: Program používa podmienku *if len(points) >= 2*, ktorá spustí výpočet len keď je splnená.
- B. Problém: Vstupný súbor môže obsahovať riadky, ktoré nie sú súradnicami bodov (napr. text atď.).
Riešenie: Prevod dát na *float* vo vnútri *try-except*, takže keď problém nastane, hneď sa odchyť a vyskočí chyba.
- C. Problém: Nevieme, či pracujeme s polygónom alebo s polylíniou, a potom by sa mohlo stať, že máme vstupný polygón a výstupnú polylíniu.
Riešenie: Pomocou definovania tried *Polygón* a *Polyline* a metódy *closed()* ktorá zistí, či je posledný a prvý bod rovnaký sa problém vyrieši. Následne sa pri polygóne novovytvorené koncové body spoja opäť do polygónu.
- D. Problém: V každej iterácii počet bodov narastá viac a viac, pokiaľ by sa teda ako počet iterácií zadalo extrémne vysoké číslo, bolo by to veľmi náročné pre pamäť alebo by sa vytvoril obrovský textový súbor. Taktiež by sa mohlo zadať záporné číslo, čo by viedlo k nezmyselnému výpočtu.
Riešenie: Metóda *run* obsahuje kontroly a tie v prípade priveľkého alebo záporného čísla hneď program zastavia a zahlásia chybu.

E. Problém: Vstupný súbor vôbec neexistuje.

Riešenie: Rieši blok s *FileNotFoundException*.

Alternatívne riešenia úlohy

Riešenie úlohy by šlo samozrejme jednoducho spraviť aj bez použitia OOP ako alternatívne riešenie. Ďalej by určite šlo použiť v riešení aj knižnicu NumPy, a súradnice spracovávať pomocou vektorových operácií. Retrospektívne by to možno bolo krajšie a určite lepšie pre väčšie množstvá bodov, každopádne táto verzia úlohy je už pri písaní tohto odstavca dokumentácie príliš ďaleko na to, aby sa menil celý jej koncept.

Vstupné dáta

Vstupom do programu je jednoduchý textový súbor, ktorý predstavuje množinu bodov v 2D priestore. Každý riadok by mal obsahovať súradnice jedného bodu vo formáte [x,y]. Program pri načítaní dáta vyčistí – odstráni hranaté zátvorky a čiarky a ďalej s nimi pracuje.

Testovacie vstupné dáta „*points.txt*“ priložené pri programe obsahujú body v rôznych formách, aby sa overila funkčnosť programu. Nachádzajú sa tam body popisujúce polygón, súradnice sú zobrazené aj v desatinných aj celý číslach a tiež sa tam nachádza prázdny riadok.

Výstupné dáta

Program generuje nový textový súbor „*chaikin.txt*“ kde je každý vypočítaný bod zapísaný na samostatnom riadku v rovnakom formáte ako vo vstupnom súbore ([x,y]). Tým pádom užívateľ môže v prípade potreby výstupné dáta kľudne nanovo načítať ako vstupné dáta.

Priebeh práce

Priebeh práce bol veľmi, veľmi zdĺhavý (vlastným pričinením). Začal sa naštudovaním konceptu Chaikinovho algoritmu a veľmi hrubým náčrtom programu, ktorý sa následne minimálne 10x zmenil. Samotný výpočtový algoritmus bol spracovaný celkom rýchlo, každopádne zvyšné prvky OOP okolo toho dali celkom zabráť, a stále si ich spracovaním nie som kompletne istá. Každopádne celé to bolo zatienené zbytočne dlhou dobou, ktorú som strávila pri testovaní rôznych sad dát a následnou nerelevantnou odbočkou k skúmaniu rôznych kriviek a vizualizácie aproximácií.

Námety na vylepšenie

Jeden z hlavných námetov na vylepšenie by určite mohla byť vizualizácia javu, teda vykreslenie aj pôvodnej aj zjemnenej výslednej krivky (napr. pomocou Matplotlib). Ďalším zaujímavým vylepšením by mohla byť úprava programu aj pre priestorové krivky alebo pridanie dodatočných výpočtov ako napríklad zmenu dĺžky krivky.

Záverečný povzdych

Úloha bola veľmi zaujímavá ako koncept a povedala by som, že asi primerane zvládnuteľná nebyť môjho vlastného zamotania v nápadoch a myšlienkach. Po solídnych pár hodinách skúmania, prepočítavania a zabárania sa do čiernej diery matematickej fascinácie témou úlohy a jej podobných tém na ten povzdych naozaj aj došlo, keď som si uvedomila, že sa na to nemôžem zamerať len z matematického hľadiska, ale musí dôjsť aj k nejakému tomu programovaniu a praktickej aplikácii. Nakoniec to však dopadlo dúfam prevažne funkčne, a aspoň mi to slúžilo ako pripomenutie toho, že tie teoretické koncepty a papierové výpočty sú predsa len moja silnejšia stránka.

Zoznam literatúry

CHAIKIN G. M. (1974): An algorithm for high-speed curve generation. Computer Graphics and Image Processing, 3, 4, 346–349.